

AIKEN

Documentación del Proyecto — Blanqueria

E-commerce de blanquería y textil para el hogar

Frontend: <https://aikenblanco.com.ar>

API (Worker Cloudflare): <https://api.aikenblanco.com.ar>

Repositorio: <https://github.com/JuanHernandezFeser/blanqueria.git>

Fecha: 30/07/2026

React 18

Vite 5

TypeScript

Tailwind CSS

shadcn/ui

Zustand

Bun

Hono

SQLite

Cloudflare Workers

Cloudflare D1

Mercado Pago

Índice

1. Visión General y Arquitectura
2. Stack Tecnológico
3. Estructura de Carpetas
4. Rutas del Frontend
5. API — Endpoints
6. Base de Datos
7. Variables de Entorno y Secrets
8. Comandos de Desarrollo
9. Flujo Git y Ramas
10. Deploy a Producción
11. Notas, Problemas Conocidos y Seguridad
12. Historial Reciente

1. Visión General y Arquitectura

AIKEN / Blanqueria es una tienda online de artículos de blanquería y textil para el hogar (sábanas, toallas, almohadas, acolchados, manteles). Cuenta con catálogo, carrito de compras, checkout con **Mercado Pago** o **transferencia bancaria**, registro de usuarios, historial de pedidos y un panel de administración completo (productos, categorías, slides, pedidos y medios de pago).

1.1 Componentes del sistema

Componente	Tecnología	Rol
Frontend (SPA)	React + Vite + TypeScript	Tienda, panel admin y experiencia de usuario. Se compila a estáticos (<code>dist/</code>) y se sirve en Cloudflare Pages.
Backend local	Bun + Hono + SQLite	API de desarrollo local (<code>server/</code>). Usa <code>bun:sqlite</code> con archivo <code>server/data/blanqueria.db</code> .
Worker Cloudflare	Cloudflare Workers + D1	API de producción (<code>workers/api</code>). Se sirve en <code>api.aikenblanco.com.ar</code> y persiste en D1 (<code>blanqueria-db</code>).
Functions de Pages	Cloudflare Pages Functions	Proxy <code>/api/*</code> hacia el Worker (<code>functions/api/[[route]].ts</code>).

1.2 Arquitectura



Nota importante: el cliente de la API en producción (`src/services/api.ts`) apunta fijo a `https://api.aikenblanco.com.ar/api`. En desarrollo, el proxy de Vite reenvía `/api` a `http://localhost:3001` (usado por `shippingService.ts`).

2. Stack Tecnológico

2.1 Frontend (`package.json` raíz)

Tecnología	Versión	Uso
React	^18.3.1	UI
Vite	^5.4.19	Build / dev server (puerto 8080)
TypeScript	^5.8.3	Lenguaje
react-router-dom	^6.30.1	Ruteo
Zustand	^5.0.11	Estado global (carrito con TTL 72h, auth, stores de catálogo)
@tanstack/react-query	^5.83.0	Fetch y cache
Tailwind CSS	^3.4.17	Estilos
shadcn/ui	Radix primitives	Componentes (50+ primitivos)
framer-motion	^12.36.0	Animaciones
react-hook-form + Zod	^7.61.1 / ^3.25.76	Formularios y validación
Recharts	^2.15.4	Gráficos
sonner	^1.7.4	Toasts
lucide-react	^0.462.0	Iconos
Vitest	^3.2.4	Tests unitarios
Playwright	^1.57.0	Tests E2E
embla-carousel-react	^8.6.0	Carruseles

2.2 Backend local (`server/package.json`)

Tecnología	Versión	Uso
Bun	runtime	Runtime + <code>bun:sqlite</code>
Hono	^4.7.0	Framework HTTP
SQLite	bun:sqlite nativo	Base de datos local
Resend	API externa	Emails transaccionales
bcrypt (Web Crypto)	nativo	Hash de contraseñas

2.3 Worker Cloudflare (`workers/api/package.json`)

Tecnología	Versión	Uso
Wrangler	^4.0.0	CLI de deploy / dev
Cloudflare Workers	runtime	API serverless
Cloudflare D1	SQLite serverless	Persistencia (binding <code>DB</code>)
@cloudflare/workers-types	^4.20250301.0	Tipos

3. Estructura de Carpetas

```
blanqueria/
├── src/                                # Frontend React (Vite)
│   ├── components/                   # Componentes
│   │   ├── shared/                  # Reutilizables (carruseles, breadcrumbs, autocomplete...)
│   │   └── ui/                      # Primitivas shadcn/ui (50+)
│   ├── data/                        # Tipos y datos (Product, categories, initialProducts)
│   ├── hooks/                      # Hooks personalizados (use-mobile...)
│   ├── lib/                        # Utilidades (cn, helpers)
│   ├── pages/                      # Páginas de la app
│   │   └── admin/                  # Sub-páginas del panel admin
│   ├── services/                   # Cliente de API y servicios (api, shippingService)
│   ├── stores/                     # Stores Zustand (cart, auth, product, category...)
│   └── test/                       # Tests (Vitest + E2E Playwright)
├── server/                          # Backend Bun + Hono (dev local)
│   ├── data/                      # Base SQLite (gitignored: blanqueria.db)
│   ├── dist/                      # Compilado
│   ├── src/
│   │   ├── routes/                # Rutas Hono (auth, products, orders...)
│   │   ├── db.ts                 # Schema + migraciones idempotentes (runSilent)
│   │   ├── index.ts              # Bootstrap + CORS + /uploads
│   │   └── uploads/              # Archivos subidos (gitignored)
├── workers/
│   ├── api/                       # Cloudflare Worker (API de producción)
│   │   ├── migrations/           # Migraciones D1 (0001_add_product_dimensions.sql)
│   │   ├── schema.sql            # Esquema base D1
│   │   ├── seed.sql              # Seed D1
│   │   ├── wrangler.toml         # Config worker + binding D1
│   │   └── src/
│   │       ├── index.ts          # Router
│   │       └── routes/           # Rutas del worker
├── functions/
│   └── api/[[route]].ts          # Pages Functions: proxy /api/* → worker
├── public/                        # Estáticos (logos, favicon)
├── imagenes-productos/           # Imágenes de productos
├── docs/                         # Guías y documentación
├── package.json                  # Scripts del frontend
├── vite.config.ts                # Dev server (8080) + proxy /api → 3001
└── migration.sql                 # Datos/migración SQLite local
```

3.1 Stores Zustand

Store	Archivo	Contenido
cartStore	<code>src/stores/cartStore.ts</code>	Items del carrito (persistido, TTL 72h)
authStore	<code>src/stores/authStore.ts</code>	Sesión JWT y perfil
productStore	<code>src/stores/productStore.ts</code>	Catálogo de productos
categoryStore	<code>src/stores/categoryStore.ts</code>	Categorías y subcategorías
ambienteStore	<code>src/stores/ambienteStore.ts</code>	Ambientes

heroStore	<code>src/stores/heroStore.ts</code>	Slides del hero
orderStore	<code>src/stores/orderStore.ts</code>	Pedidos del usuario
bankConfigStore	<code>src/stores/bankConfigStore.ts</code>	Config de transferencia

4. Rutas del Frontend

Definidas en `src/App.tsx` (React Router v6).

Ruta	Componente	Descripción
<code>/</code>	Home	Landing: hero carousel, categorías destacadas, carruseles de productos nuevos/destacados
<code>/catalogo</code>	Catalog	Búsqueda, filtros (categoría/subcategoría/marca/precio/stock), ordenamiento, panel lateral de producto
<code>/producto/:id</code>	ProductDetail	Detalle de producto (galería, variantes, colores, stock)
<code>/carrito</code>	Cart	Carrito + calculadora de envío + resumen
<code>/checkout</code>	Checkout	3 pasos: envío → pago (MP/transferencia) → confirmación
<code>/pago/retorno</code>	PaymentReturn	Retorno post-MercadoPago (crea la orden y muestra resultado)
<code>/login</code>	Login	Inicio de sesión
<code>/registro</code>	Register	Registro de usuario
<code>/mi-cuenta</code>	MyAccount	Perfil + historial de pedidos
<code>/completar-perfil</code>	CompleteProfile	Completar datos de usuario
<code>/verificar-email/:token</code>	VerifyEmail	Verificación de email
<code>/admin</code>	Admin	Panel de administración (tabs: Productos, Categorías, Hero, Pedidos, Pagos, Ambientes)
<code>/faq</code>	FAQ	Preguntas frecuentes
<code>*</code>	NotFound	404

5. API — Endpoints

Existen **dos implementaciones** del backend con la misma interfaz. La de producción es el Worker de Cloudflare (`workers/api`); la local (`server/`, Bun + Hono) se usa para desarrollo.

Método	Ruta	Auth	Admin	Descripción
GET	<code>/api/health</code>	-	-	Health check
POST	<code>/api/auth/register</code>	-	-	Registrar usuario
POST	<code>/api/auth/login</code>	-	-	Login → JWT
GET	<code>/api/auth/me</code>	Sí	-	Perfil actual
PUT	<code>/api/auth/profile</code>	Sí	-	Actualizar perfil
POST	<code>/api/auth/verify-email</code> / <code>/resend-verification</code>	-	-	Verificación de email
GET	<code>/api/products</code>	-	-	Listar productos
GET	<code>/api/products/:id</code>	-	-	Producto por ID
POST	<code>/api/products</code>	Sí	Sí	Crear producto
PUT	<code>/api/products/:id</code>	Sí	Sí	Actualizar producto (incluye weight/width/height/length)
DELETE	<code>/api/products/:id</code>	Sí	Sí	Eliminar producto
GET/POST/PUT/DELETE	<code>/api/categories</code> ...	Sí*	Sí	CRUD categorías (subcategorías en JSON)
GET/POST/PUT/DELETE	<code>/api/ambientes</code> ...	Sí*	Sí	CRUD ambientes
GET/POST/PUT/DELETE	<code>/api/hero-slides</code> ...	Sí*	Sí	CRUD slides del hero
POST	<code>/api/hero-slides/reorder</code>	Sí	Sí	Reordenar slides (batch)
GET	<code>/api/orders</code>	Sí	-	Listar órdenes (propias, o todas si admin)
POST	<code>/api/orders</code>	-	-	Crear orden (guest)
PATCH	<code>/api/orders/:id/status</code>	Sí	Sí	Cambiar estado de pedido
GET/PUT	<code>/api/bank-config</code>	Sí*	Sí	Config transferencia bancaria (GET público)
POST	<code>/api/create-preference</code>	-	-	Crear preferencia Mercado Pago
POST	<code>/api/webhooks/mercadopago</code>	-	-	Webhook MP
POST	<code>/api/upload</code>	Sí	Sí	Subir imágenes (max 10MB, admin)
POST	<code>/api/shipping/quote</code>	-	-	Cotizar envío (paquetes con weight/width/height/length)
GET	<code>/uploads/:file</code>	-	-	Servir archivos (solo backend local)

POST	<code>/api/testing/reset</code>	-	-	Reset de datos de prueba (solo dev, backend local)
------	---------------------------------	---	---	--

* Los métodos de escritura requieren rol admin; los GET de lectura son públicos. La autenticación se envía como header

`Authorization: Bearer <JWT>`.

5.1 Campos del producto relevantes para envío

El producto soporta **peso, alto, ancho y largo** (`weight`, `height`, `width`, `length`), editables desde el panel admin. Se usan en `buildShipmentPackages()` (`src/services/shippingService.ts`) con valores por defecto 1 kg y 20×20×20 cm.

6. Base de Datos

6.1 Dos motores

	Local (dev)	Producción
Motor	SQLite vía <code>bun:sqlite</code>	Cloudflare D1
Ubicación	<code>server/data/blanqueria.db</code>	D1 <code>blanqueria-db</code> (id <code>f0b2dc3d-a6b2-4174-ae9c-1750eef96eee</code>)
Schema	<code>server/src/db.ts</code> (migraciones idempotentes con <code>runSilent</code>)	<code>workers/api/schema.sql</code> (base) + <code>workers/api/migrations/</code>
Seed	<code>server/src/seed.ts</code>	<code>workers/api/seed.sql</code>

6.2 Tabla `products`

Columna	Tipo	Descripción
id	TEXT PK	Identificador (timestamp)
name	TEXT	Nombre
description	TEXT	Descripción
brand	TEXT	Marca
category	TEXT	Categoría
subcategory	TEXT	Subcategoría
price	REAL	Precio
stock	INTEGER	Stock simple
image	TEXT	Imagen principal
images_json	TEXT	Galería (array JSON)
variants_json	TEXT	Variantes/talles (array JSON)
colors_json	TEXT	Colores (array JSON)
variant_stock_json	TEXT	Stock por variante/color (objeto JSON)
ambientes_json	TEXT	Ambientes (array JSON)
weight	REAL	Peso (kg) — para envío
width	REAL	Ancho (cm) — para envío
height	REAL	Alto (cm) — para envío
length	REAL	Largo (cm) — para envío
featured	INTEGER	Destacado (0/1)
is_new	INTEGER	Nuevo (0/1)
created_at / updated_at	TEXT	Fechas

6.3 Otras tablas

Tabla	Columnas principales
users	email, password_hash, name, is_admin, phone, address, locality, province, postal_code, email_verified, verification_token
categories	name, image, description, subcategories_json, icon
ambientes	name, image, description
hero_slides	type, image, product_id, title, subtitle, link, order, video_url
orders	customer_name/email, date, subtotal, shipping_cost, total, order_status, payment_method, payment_status, items_json, shipping_address_json, source
bank_config	bank_name, cbu, alias, account_holder, discount_percentage
d1_migrations	Solo en D1: control de migraciones aplicadas (name, applied_at)

6.4 Sistema de migraciones D1

Configurado en `workers/api/wrangler.toml` con `migrations_dir = "migrations"`. Cada cambio de schema se agrega como un archivo numerado:

Migración	Contenido
<code>0001_add_product_dimensions.sql</code>	Agrega <code>weight</code> , <code>width</code> , <code>height</code> , <code>length</code> a <code>products</code>

Atención: en la D1 de producción la migración 0001 ya está aplicada manualmente y registrada en `d1_migrations`. `schema.sql` es la línea base sin esas columnas; no incluye `ALTER TABLE` porque D1 no los agrega a tablas existentes.

7. Variables de Entorno y Secrets

7.1 Backend local — `server/.env`

Variable	Ejemplo	Descripción
PORT	3001	Puerto del server Hono
JWT_SECRET	blanqueria-secret-key-change-in-production	Secreto JWT (HS256)
DB_PATH	./data/blanqueria.db	Ruta de la SQLite local
MERCADOPAGO_ACCESS_TOKEN		Access token de Mercado Pago
MERCADOPAGO_PUBLIC_KEY		Public key de Mercado Pago
MERCADOPAGO_WEBHOOK_SECRET		Secreto del webhook
SITE_URL	http://localhost:8080	URL del sitio (links en emails)
RESEND_API_KEY		Key de Resend para emails
EMAIL_FROM	onboarding@resend.dev	Remitente de emails
TEST_RESET_KEY	dev-reset-key	Key del endpoint de reset (solo dev)

7.2 Secrets del Worker — `wrangler secret put <NOMBRE>`

Se setean desde el dashboard de Cloudflare o con wrangler (carpeta `workers/api`):

```
wrangler secret put JWT_SECRET
wrangler secret put RESEND_API_KEY
wrangler secret put MERCADOPAGO_ACCESS_TOKEN
wrangler secret put MERCADOPAGO_PUBLIC_KEY
wrangler secret put MERCADOPAGO_WEBHOOK_SECRET
wrangler secret put IMGBB_API_KEY
wrangler secret put MICORREO_CUSTOMER_ID
wrangler secret put MICORREO_EMAIL
wrangler secret put MICORREO_PASSWORD
```

7.3 Vars del Worker — `wrangler.toml` [vars]

Variable	Valor
SITE_URL	https://aikenblanco.com.ar
EMAIL_FROM	tienda@aikenblanco.com.ar

8. Comandos de Desarrollo

8.1 Instalación

```
npm install          # dependencias del frontend (raíz)
cd server && bun install
cd workers/api && npm install
```

8.2 Frontend (raíz)

Comando	Qué hace
<code>npm run dev</code>	Dev server Vite en <code>http://localhost:8080</code> (proxy <code>/api</code> → <code>:3001</code>)
<code>npm run dev:server</code>	Backend Bun con watch (<code>server/src/index.ts</code>)
<code>npm run dev:all</code>	Frontend + backend juntos (concurrently)
<code>npm run build</code>	Build de producción a <code>dist/</code>
<code>npm run build:dev</code>	Build en modo development
<code>npm run lint</code>	ESLint sobre todo el proyecto
<code>npm run preview</code>	Previsualiza el build local
<code>npm run test</code> / <code>test:watch</code>	Tests Vitest

8.3 Backend local

```
cd server
bun dev          # bun --watch src/index.ts  (puerto 3001)
bun start        # bun src/index.ts
```

8.4 Worker Cloudflare

```
cd workers/api
npm run dev      # wrangler dev (worker local con D1 local)
npm run deploy   # wrangler deploy (producción)
npm run db:init  # wrangler d1 execute --remote --file=schema.sql
npm run db:seed  # wrangler d1 execute --remote --file=seed.sql
```

8.5 Tests

- **Unitarios (Vitest):** `npm run test` — `src/test/`
- **E2E (Playwright):** `npx playwright test` — `src/test/e2e/checkout-transferencia.spec.ts` (config en `playwright.config.ts`)

9. Flujo Git y Ramas

Hay dos ramas principales:

Rama	Rol
<code>local</code>	Rama de trabajo/desarrollo (activa por defecto)
<code>main</code>	Rama de producción

Remote: `origin` → <https://github.com/JuanHernandezFeser/blanqueria.git>

9.1 Comandos habituales

```
# Estado y log
git branch --show-current
git status
git log --oneline -10

# Trabajar en local
git add <archivos>
git commit -m "Descripcion clara del cambio"
git push origin local

# Pasar a producción
git checkout main
git merge local
git push origin main
git checkout local
```

10. Deploy a Producción

10.1 Frontend — Cloudflare Pages

El frontend se compila con Vite y se sube a Cloudflare Pages (dominio `aikenblanco.com.ar`). La carpeta `functions/` incluye un proxy que reenvía `/api/*` al Worker.

```
# 1. Compilar el frontend (genera dist/)
npm run build

# 2. Subir a Cloudflare Pages
npx wrangler pages deploy dist
```

Si Pages está conectado al repo por integración de GitHub, el deploy puede ser automático al pushear a `main`. El comando anterior es la vía manual.

10.2 Worker — Cloudflare Workers

```
cd workers/api
npx wrangler deploy
```

Publica el worker `blanqueria-api` en `https://api.aikenblanco.com.ar`. Verificar que aparezca `Uploaded` y `Deployed` sin errores.

10.3 Base de datos — D1

```
cd workers/api

# Aplicar migraciones pendientes (producción)
npx wrangler d1 migrations apply blanqueria-db --remote

# Inicializar schema base (solo DB nueva)
npm run db:init

# Sembrar datos iniciales (solo DB nueva)
npm run db:seed

# Inspección rápida
npx wrangler d1 execute blanqueria-db --remote --command "SELECT name FROM
pragma_table_info('products');"

```

10.4 Secrets (si cambiaron)

```
cd workers/api
wrangler secret put NOMBRE_DEL_SECRET
```

10.5 Deploy completo (copy-paste)

```
# 1. Verificar rama de trabajo
git branch --show-current      # debe decir: local

# 2. Commit y push de la rama local
git add .
git commit -m "Descripcion clara del cambio"
git push origin local

# 3. Merge a main y push
git checkout main
git merge local
git push origin main
git checkout local

# 4. Deploy frontend (Pages)
npm run build
npx wrangler pages deploy dist

# 5. Deploy worker
cd workers/api
npx wrangler deploy

# 6. Aplicar migraciones D1 (si corresponde)
npx wrangler d1 migrations apply blanqueria-db --remote
```


11. Notas, Problemas Conocidos y Seguridad

Tema	Estado
JWT_SECRET hardcodeado en <code>.env.example</code> (local)	Pendiente: usar un valor fuerte en producción
Mercado Pago en dev	En desarrollo simula pago aprobado (intencional)
Mercado Pago en producción	Requiere credenciales reales probadas
API client hardcodeado	<code>src/services/api.ts</code> apunta fijo a producción; en dev no se puede apuntar a local sin editarlo
Base de datos	SQLite local no escala a múltiples instancias; en producción se usa D1 (serverless)
Secrets	Nunca commitear <code>.env</code> ni keys al repo (ya cubierto por <code>.gitignore</code>)
Migraciones D1	Los cambios de schema se hacen con migraciones (0001 ya aplicada en producción)
Tests	Solo hay unit tests de ejemplo y una suite E2E de checkout

Advertencia de seguridad: no subir `.env`, keys de Resend/MercadoPago ni secrets al repositorio o a repos públicos.

12. Historial Reciente

```
ea9a980 Mail de aviso interno al recibir pedido nuevo + FAQ en navbar
76a54b9 commit all local changes
2fc4d48 Agregar guia de deploy en markdown y PDF
02d5cfb Fix upload: chunked base64 encoding para evitar 502 en archivos grandes
df26f68 Quitar accesos demo, logo MP oficial, fix pedidos en produccion, actualizar dominios
9048a26 Completar suite E2E de checkout: 28 tests, fixes de validacion
f738f0e Agregar tests E2E de checkout, validación de email/telefono
11c036c Agregar backend Hono, workers Cloudflare y nuevas features de frontend
d6da991 Agregar .env al gitignore
```

12.1 Cambios recientes de infraestructura

- **2026-07-30** — Configurado sistema de migraciones D1 (`migrations_dir`) y migración `0001_add_product_dimensions.sql`. Agregadas las columnas `weight`, `width`, `height`, `length` a la tabla `products` de la D1 de producción.